

MICROPROCESSOR & EMBEDDED SYSTEMS

UNIT 1 — Complete Question Bank

Detailed 10-Mark Answers

This document contains **detailed, exam-ready answers** for every question in the Unit 1 Question Bank. Each answer is written to score maximum marks in a 10-mark university exam. It covers architecture, registers, flags, addressing modes, instruction set, ALPs, and assembler directives.

Subject	Microprocessor and Embedded Systems
Unit	Unit 1 — 8086 Architecture & Programming
Total Questions	30 Questions (all answered)
Answer Style	Detailed 10-Mark University Answers
Topics Covered	Architecture, Flags, Addressing, Instructions, ALPs, Directives

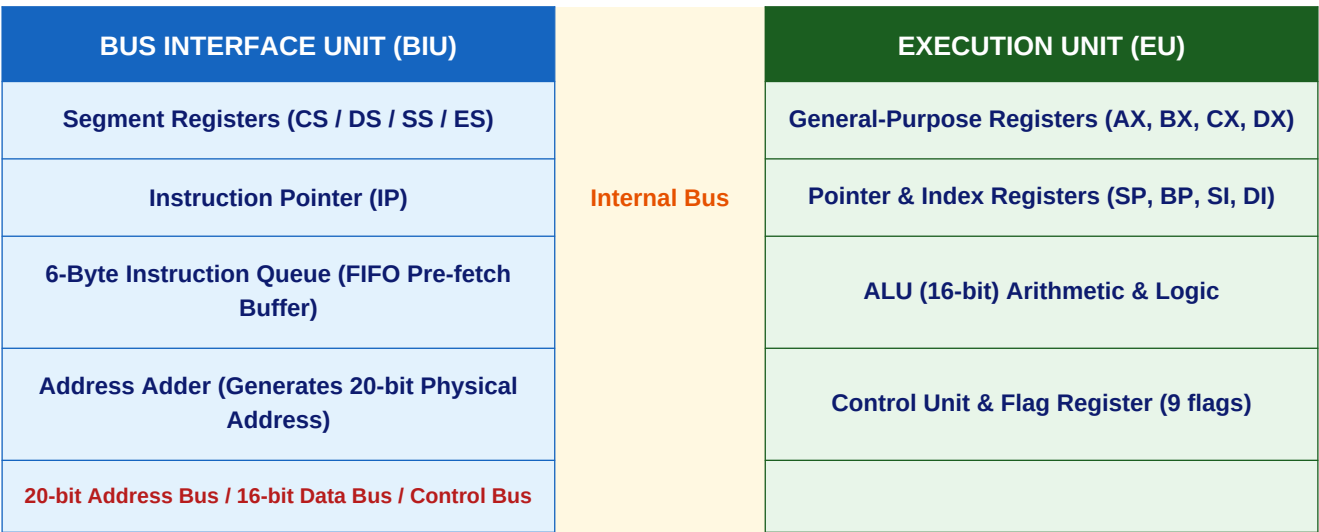
Q1. With a Block diagram explain the architecture of 8086 microprocessor.

[10 Marks]

Introduction

The Intel 8086 is a **16-bit microprocessor** introduced in 1978 as the third-generation microprocessor. It uses **HMOS technology** and has a **20-bit address bus** enabling it to address up to **1 Megabyte ($2^{20} = 1,048,576$ bytes)** of memory. Internally, the 8086 is divided into two independent functional units — the **Bus Interface Unit (BIU)** and the **Execution Unit (EU)** — that operate in parallel using a pipeline mechanism.

Block Diagram — 8086 Internal Architecture



1. Bus Interface Unit (BIU)

The BIU handles all **external bus operations** — it sends addresses, fetches instructions, reads/writes data. It has the following sub-components:

Component	Description
Segment Registers (CS,DS,SS,ES)	Hold 16-bit base addresses of the four memory segments
Instruction Pointer (IP)	16-bit register; holds offset of next instruction within code segment
6-Byte Instruction Queue	Pre-fetches up to 6 instruction bytes from memory (FIFO); enables pipelining
Address Adder	Computes 20-bit physical address: Segment × 10H + Offset
Bus Control Logic	Generates address, data, and control bus signals; manages memory/IO access

2. Execution Unit (EU)

The EU **decodes and executes** the instructions fetched by the BIU from the instruction queue:

Component	Description
ALU (16-bit)	Performs all arithmetic (add, sub, mul, div) and logical (AND, OR, XOR, NOT) operations
Control Unit	Decodes instruction opcode and generates internal control signals

Flag Register	16-bit register; 9 bits used as condition/control flags (CF, PF, AF, ZF, SF, OF, DF, IF, TF)
General Registers (AX,BX,CX,DX)	Four 16-bit data registers; each split into 8-bit high/low halves (e.g., AH/AL)
Pointer & Index Registers	SP (Stack Pointer), BP (Base Pointer), SI (Source Index), DI (Destination Index)

3. Pipelining

Pipelining: While the EU executes the current instruction, the BIU simultaneously fetches the next instruction bytes into the queue. This overlapping doubles throughput. The queue holds up to 6 bytes. If the EU executes a branch/jump, the queue is flushed and re-filled from the new address.

4. Memory Segmentation & Physical Address

The 8086 uses a **segmented memory model**. The 20-bit physical address is formed as:

Physical Address = Segment Register × 10H + Effective Address (Offset)

Example: DS = 2000H, EA = 5A00H → PA = 20000H + 5A00H = 25A00H

5. Key Specifications

Parameter	Value	Parameter	Value
Data Bus	16 bits	Address Bus	20 bits
Memory Space	1 MB (2 ²⁰)	Technology	HMOS
Registers	14 × 16-bit	Instruction Queue	6 bytes (FIFO)
Clock Speed	5–10 MHz	Generation	3rd (1978)

Q2. Indicate the addressing modes of the destination operand and calculate its physical address for: i) ADD [DI],AL ii) SUB BLOCK,AX iii) AND [BX+DI],CL iv) ADC [BP+100H],AX

[10 Marks]

Given Register Values

Register	Value	Register	Value	Register	Value
CS	1000H	DS	2000H	ES	3000H
SS	4000H	BX	3FFFH	DI	5A00H
SP	64A0H	IP	A000H	BP	69B0H
BLOCK	4000H	—	—	—	—

Physical Address Formula

Physical Address (PA) = Segment Register × 10H + Effective Address (EA)

Default segments: DS for [BX],[SI],[DI],[BX+DI] etc. | SS for [BP],[SP],[BP+disp]

i) ADD [DI], AL

Addressing Mode: Register Indirect Addressing

The destination operand [DI] means the effective address is the content of the DI register. This is Register Indirect addressing because the address is held in a register (DI), not given directly in the instruction.

- Effective Address (EA) = DI = 5A00H
- Default Segment = DS = 2000H
- PA = DS × 10H + EA = 20000H + 5A00H = **25A00H**

ii) SUB BLOCK, AX

Addressing Mode: Direct Addressing (Memory Direct)

BLOCK is a symbolic name for a memory address (4000H). The effective address is given directly in the instruction as a 16-bit displacement. This is Direct Addressing.

- Effective Address (EA) = BLOCK = 4000H
- Default Segment = DS = 2000H
- PA = DS × 10H + EA = 20000H + 4000H = **24000H**

iii) AND [BX+DI], CL

Addressing Mode: Based Indexed Addressing

The destination address is computed by adding the contents of a Base register (BX) and an Index register (DI). This is Based Indexed Addressing.

```
3FFFH
+ 5A00H
-----
```

99FFH

- $EA = BX + DI = 3FFFFH + 5A00H = 99FFH$
- $PA = DS \times 10H + EA = 20000H + 99FFH = \mathbf{299FFH}$

iv) ADC [BP+100H], AX

Addressing Mode: Based Addressing with Displacement

$EA = BP + 100H$. Note: BP uses SS as default segment (not DS).

```
69B0H
+ 0100H
-----
6AB0H
```

- $EA = BP + 100H = 69B0H + 0100H = 6AB0H$
- Default Segment = SS = 4000H (BP always uses SS!)
- $PA = SS \times 10H + EA = 40000H + 6AB0H = \mathbf{46AB0H}$

Summary Table

Instruction	Mode	EA	Segment Used	Physical Address
ADD [DI], AL	Register Indirect	5A00H	DS=2000H	25A00H
SUB BLOCK, AX	Direct	4000H	DS=2000H	24000H
AND [BX+DI], CL	Based Indexed	99FFH	DS=2000H	299FFH
ADC [BP+100H], AX	Based w/ Displacement	6AB0H	SS=4000H	46AB0H

Q3. Write an assembly level program to search an element in a given array of bytes using the linear search algorithm.

[10 Marks]

Algorithm — Linear Search

Linear search checks each element of the array one by one from the beginning until the key element is found or the end of the array is reached.

Step	Action
1	Load the key (element to search) into AL register
2	Point SI to the beginning of the array
3	Load the count (size of array) into CX register
4	Compare AL with current element [SI]
5	If equal → element found; store and display position
6	If not equal → increment SI, decrement CX, repeat from step 4
7	If CX = 0 and not found → display "Element Not Found"

Complete Assembly Language Program

```
DATA SEGMENT
    ARR    DB  10H, 20H, 30H, 40H, 50H  ; Array of 5 bytes
    N      DB  05H                      ; Number of elements
    KEY    DB  30H                      ; Element to search
    POS    DB  00H                      ; Stores position (1-based)
    MSG1   DB  "Element Found at Position: $"
    MSG2   DB  "Element Not Found!$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AL, KEY
    LEA SI, ARR
    MOV CL, N
    MOV CH, 00H
    MOV BL, 01H                ; Position counter

SEARCH:
    CMP AL, [SI]
    JE  FOUND
    INC SI
    INC BL
    DEC CX
    JNZ SEARCH

NOT_FOUND:
    LEA DX, MSG2
    MOV AH, 09H
```

```

        INT 21H
        JMP DONE
FOUND:
        MOV POS, BL
        LEA DX, MSG1
        MOV AH, 09H
        INT 21H
        MOV DL, BL
        ADD DL, 30H
        MOV AH, 02H
        INT 21H
DONE:
        MOV AH, 4CH
        INT 21H
CODE ENDS
END START

```

Program Trace

Iteration	SI points to	AL (key)	[SI] value	CMP Result	Action
1	ARR[0]	30H	10H	Not Equal	INC SI, INC BL, DEC CX
2	ARR[1]	30H	20H	Not Equal	INC SI, INC BL, DEC CX
3	ARR[2]	30H	30H	Equal	Jump to FOUND, POS=3

Output: Element Found at Position: 3

Q4. Illustrate the following Assembler directives with proper examples:

[10 Marks]

i) ASSUME ii) OFFSET iii) EQU iv) MACRO

What are Assembler Directives?

Assembler directives (also called pseudo-instructions) are commands given to the assembler to control how the source code is assembled. They do **NOT generate machine code instructions**. They help define data, organize memory segments, and make the code modular.

i) ASSUME Directive

Purpose: Tells the assembler which logical segment name corresponds to which segment register (CS, DS, SS, ES).

- Must appear at the beginning of the CODE segment, before any instruction.
- It does not physically load the segment address into the register — you still need MOV AX, DATA / MOV DS, AX to initialize DS at runtime.

```
DATA SEGMENT
    NUM DB 25H
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA, SS:STACK
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AL, NUM          ; Assembler knows NUM is in DS (ASSUME DS:DATA)
CODE ENDS
END START
```

ii) OFFSET Directive

Purpose: OFFSET returns the **16-bit offset address** of a variable, label, or procedure. It is used to load a pointer to a data item.

- OFFSET gives the address OF a variable, not its value.
- Equivalent to LEA instruction: MOV SI, OFFSET ARR = LEA SI, ARR

```
DATA SEGMENT
    PRICES DB 10H, 20H, 30H, 40H, 50H
    MSG     DB "HELLO$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV SI, OFFSET PRICES ; SI = offset of PRICES from start of DS
    MOV AL, [SI]          ; AL = PRICES[0] = 10H
    MOV DX, OFFSET MSG    ; DX = address of "HELLO$"
    MOV AH, 09H
```



```

        INT 21H                ; Displays: HELLO
CODE ENDS
END START

```

iii) EQU (Equate) Directive

Purpose: EQU gives a **symbolic name** to a constant value. Like **#define** in C language. EQU does NOT allocate any memory.

```

COUNT EQU 10                ; COUNT = 10
CR EQU 0DH                   ; Carriage Return
LF EQU 0AH                   ; Line Feed

DATA SEGMENT
    NEWLINE DB CR, LF, "$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV CX, COUNT            ; CX = 10
LOOP1:
    LOOP LOOP1
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START

```

iv) MACRO Directive

Purpose: A MACRO defines a **named block of assembly instructions** that can be called multiple times with different parameters. The assembler replaces each macro call with the full macro body (inline expansion).

```

DISPLAY MACRO MSG_NAME
    LEA DX, MSG_NAME
    MOV AH, 09H
    INT 21H
ENDM

DATA SEGMENT
    MSG1 DB "Hello, World!$"
    MSG2 DB "Welcome to 8086!$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    DISPLAY MSG1
    DISPLAY MSG2

```

```
MOV AH, 4CH
INT 21H
CODE ENDS
END START
```

Comparison: Macro vs Procedure

Feature	MACRO	PROCEDURE (PROC)
Expansion	Inline (at assembly time)	Separate code block (linked at run time)
Speed	Faster (no CALL/RET overhead)	Slightly slower (CALL/RET push/pop)
Code size	Grows with each call	Same code reused for each call
Parameters	Yes, supported directly	Via registers or stack

**Q5. Explain the following instructions with appropriate examples: i)MOV
ii)PUSH iii)IN iv)XCHG v)IMUL vi)POP**

[10 Marks]

Overview

These are **Data Transfer Instructions** (MOV, PUSH, POP, IN, XCHG) and an **Arithmetic Instruction** (IMUL). Data transfer instructions generally do **NOT affect flags** (except POPF).

i) MOV — Move (Copy) Data

Syntax: MOV destination, source

Description: Copies data from source to destination. Source is unchanged. Both operands must be the same size. Cannot MOV memory to memory directly; cannot load a segment register with an immediate value.

Examples:

```
MOV AX, 1234H ; AX = 1234H
```

```
MOV AL, BH ; AL = BH
```

```
MOV [BX], CX ; Memory[BX] = CX
```

```
MOV DS, AX ; DS = AX
```

Flags: MOV does NOT affect any flags.

ii) PUSH — Push onto Stack

Syntax: PUSH source

Description: Decrements SP by 2, then copies the 16-bit source operand to the top of the stack (SS:[SP]).

Examples:

```
PUSH AX ; SP=SP-2; SS:[SP] = AX
```

```
PUSH BX ; SP=SP-2; SS:[SP] = BX
```

```
PUSH [1000H] ; SP=SP-2; SS:[SP] = word at DS:[1000H]
```

Flags: PUSH does NOT affect any flags. Always pushes 16-bit values.

iii) IN — Input from Port

Syntax: IN accumulator, port

Description: Reads data from an I/O port into AL (byte) or AX (word). If port ≤ FFH: port address written directly. If port > FFH: port address must be in DX.

Examples:

```
IN AL, 0C8H ; AL = byte from I/O port 0C8H
```

```
IN AX, 34H ; AX = word from I/O port 34H
```

```
MOV DX, 0FF78H / IN AL, DX ; AL = byte from port FF78H
```

Flags: IN does NOT affect flags. Only AL or AX can receive port data.

iv) XCHG — Exchange

Syntax: XCHG operand1, operand2

Description: Swaps the contents of two operands atomically. Both operands must be the same size. Cannot exchange two memory operands directly.

Examples:

```
XCHG AX, BX ; AX and BX swap
```

```
XCHG AL, BH ; AL and BH swap
```

```
XCHG AX, [BX] ; AX and memory word at DS:[BX] swap
```

Flags: XCHG does NOT affect any flags.

v) IMUL — Integer (Signed) Multiply

Syntax: IMUL source

Description: Performs signed multiplication. For byte: $AX = AL \times \text{source}$. For word: $DX:AX = AX \times \text{source}$.

Examples:

```
IMUL BH ; AL=45H, BH=0EH ; AX = 45H × 0EH = 03C6H = +966
```

```
IMUL BH ; AL=E4H, BH=3BH ; AX = F98CH = -1652 (negative result)
```

Flags: CF=OF=1 if upper half is NOT a sign extension; else CF=OF=0.

vi) POP — Pop from Stack

Syntax: POP destination

Description: Reads the 16-bit word from the top of the stack (SS:[SP]) into the destination, then increments SP by 2. Cannot POP into CS register.

Examples:

```
POP AX ; AX = SS:[SP]; SP=SP+2
```

```
POP BX ; BX = SS:[SP]; SP=SP+2
```

```
POP [1000H] ; DS:[1000H] = SS:[SP]; SP=SP+2
```

Flags: POP does NOT affect flags (POPF is the exception).

Q6. Illustrate the flag register of 8086 microprocessor with a neat diagram.

[10 Marks]

Introduction to Flag Register

The 8086 has a **16-bit Flag Register** (also called the Processor Status Word — PSW). Out of 16 bits, **only 9 bits are used**. Flags fall into two categories: **Status Flags** (set by arithmetic/logic operations) and **Control Flags** (set by programmer to control processor behaviour).

Flag Register — Bit Layout (16-bit)

15	14	13	12	11 OF	10 DF	9 IF	8 TF	7 SF	6 ZF	5	4 AF	3	2 PF	1	0 CF
—	—	—	—	OF	DF	IF	TF	SF	ZF	—	AF	—	PF	—	CF

Status Flags — Detailed Description

Flag	Bit	Full Name	Set to 1 when...	Example
CF	0	Carry Flag	Carry out of MSB in addition OR borrow into MSB for subtraction	ADD AL, 100H → CF=1
PF	2	Parity Flag	Lower 8 bits of result have an EVEN number of 1s	MOV AL, 0110 (two 1s) → PF=1
AF	4	Auxiliary Carry Flag	Carry from bit 3 to bit 4 (lower nibble to upper nibble)	Used in BCD decimal adjust (DAA,AAA)
ZF	6	Zero Flag	Result of the operation is exactly zero	SUB AX, AX → AX=0 → ZF=1
SF	7	Sign Flag	Most Significant Bit (MSB) of result is 1 (negative if signed)	INC AX has MSB=1 → SF=1
OF	11	Overflow Flag	Signed arithmetic overflow: result too large/small for destination	ADD AL, 80H (127+1=128) → OF=1

Control Flags — Detailed Description

Flag	Bit	Full Name	When SET (=1)	When CLEARED (=0)	Set/Clear With
DF	10	Direction Flag	String instructions auto-DECREMENT	String instructions auto-INCREMENT	STC (set), CMC (clear)
IF	9	Interrupt Enable Flag	Processor ACCEPTS external maskable interrupts	External maskable interrupts are IGNORED	STI (set), CLI (clear)
TF	8	Trap Flag	Processor executes ONE instruction in debug mode	Normal instruction type execution	DT (set), DB (clear) for single-step debugging

Memory Trick: "O D I T S Z A P C" — Status flags: CF(0), PF(2), AF(4), ZF(6), SF(7), OF(11) | Control: DF(10), IF(9), TF(8)

Q7. Write an Assembly Level Program to obtain the average of two numbers.

[10 Marks]

Concept & Formula

Average of two numbers A and B = $(A + B) / 2$

In 8086: Load both numbers, add them (handling carry with ADC), then divide by 2.

Complete Assembly Language Program

```
DATA SEGMENT
    NUM1 DB 50H          ; First number = 80 decimal
    NUM2 DB 30H          ; Second number = 48 decimal
    RESULT DB ?          ; Stores the average
    MSG DB "Average = $"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AL, NUM1          ; AL = 50H
    MOV AH, 00H           ; Clear AH
    ADD AL, NUM2          ; AL = 50H + 30H = 80H
    ADC AH, 00H           ; AH = AH + CF (catches carry if sum > FFH)
    MOV BL, 02H           ; Divisor = 2
    DIV BL                ; AX / BL: quotient in AL, remainder in AH
    MOV RESULT, AL        ; Store average
    LEA DX, MSG
    MOV AH, 09H
    INT 21H
    MOV DL, AL
    ADD DL, 30H
    MOV AH, 02H
    INT 21H
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Verification: NUM1 = 50H = 80 decimal, NUM2 = 30H = 48 decimal

Sum = 80 + 48 = 128 decimal = 80H

Average = 128 / 2 = **64 decimal = 40H**

Q8. Describe the logical shift and rotate instructions with syntax. (Shift & Rotate Instructions)

[10 Marks]

Introduction

Shift and rotate instructions move bits within a register or memory operand. They are used for multiplication/division by powers of 2, bit manipulation, and serial data processing. The **Carry Flag (CF)** is always involved.

Syntax: instruction destination, count (if count > 1: load into CL first)

A) Logical Shift Instructions

1. SHL/SAL (Shift Logical/Arithmetic Left)

$CF \leftarrow MSB \leftarrow [bits] \leftarrow 0$ (zero fills from right). Each left shift multiplies by 2.

```
MOV AL, 01000101B ; AL = 45H = 69
SHL AL, 1          ; AL = 10001010B = 8AH = 138, CF = 0
```

2. SHR (Shift Logical Right)

$0 \rightarrow MSB \rightarrow [bits] \rightarrow LSB \rightarrow CF$. Used for UNSIGNED division by 2.

```
MOV AL, 10001010B ; AL = 8AH = 138
SHR AL, 1          ; AL = 01000101B = 45H = 69 (÷ 2), CF = 0
```

3. SAR (Shift Arithmetic Right)

$MSB \rightarrow MSB \rightarrow [bits] \rightarrow LSB \rightarrow CF$ (MSB is duplicated/sign extended). Used for SIGNED division by 2.

```
MOV AL, 10001010B ; AL = 8AH = -118 (signed negative)
SAR AL, 1          ; AL = 11000101B = C5H = -59, MSB stays 1 (sign preserved!)
```

B) Rotate Instructions

4. ROL (Rotate Left)

```
MOV AL, 10110001B
ROL AL, 1 ; AL = 01100011B = 63H, CF = 1 (old MSB)
```

5. ROR (Rotate Right)

```
MOV AL, 10110001B
ROR AL, 1 ; AL = 11011000B = D8H, CF = 1 (old LSB)
```

6. RCL (Rotate Left through Carry)

9-bit rotation: Old CF $\rightarrow$ LSB, old MSB $\rightarrow$ new CF.

```
; CF = 0 before
MOV AL, 10110001B
RCL AL, 1 ; Old CF(0) $\rightarrow$ LSB; MSB(1) $\rightarrow$ CF; AL = 01100010B, CF=1
```

7. RCR (Rotate Right through Carry)

9-bit rotation: Old CF → MSB, old LSB → new CF.

```
; CF = 1 before  
MOV AL, 10110001B  
RCR AL, 1          ; Old CF(1)→MSB; LSB(1)→CF; AL = 11011000B, CF=1
```

Summary Table

Instruction	Type	CF Gets	New Bit Inserted	Use For
SHL/SAL	Logical/Arith Left	Old MSB	0 into LSB	Unsigned/Signed $\times 2$
SHR	Logical Right	Old LSB	0 into MSB	Unsigned $\div 2$
SAR	Arithmetic Right	Old LSB	Sign bit into MSB	Signed $\div 2$
ROL	Circular Left	Old MSB	Old MSB into LSB	Bit testing/cycling
ROR	Circular Right	Old LSB	Old LSB into MSB	Bit testing/cycling
RCL	9-bit Rotate Left	Old MSB	Old CF into LSB	17-bit/32-bit shifts
RCR	9-bit Rotate Right	Old LSB	Old CF into MSB	17-bit/32-bit shifts

Q9. Explain the concept of segmented memory and different types of segment registers available in 8086. Show with an example how physical address is generated from logical address.

[10 Marks]

Why Segmented Memory?

The 8086 has **16-bit registers** but needs to address **1 MB (20-bit addresses)**. A 16-bit register can only directly address 64 KB. To resolve this, the 8086 uses **segmentation**: the 1 MB memory space is divided into overlapping logical units called **segments**, each up to 64 KB in size.

The Four Segment Registers

Register	Segment Name	Stores	Default For
CS	Code Segment	Base address of the code (program instructions)	All instruction fetches; IP is the offset within CS
DS	Data Segment	Base address of the main data area	Data references using BX, SI, DI, direct addressing
SS	Stack Segment	Base address of the stack area	Stack operations (PUSH, POP); SP and BP are offsets into SS
ES	Extra Segment	Base address of an additional data area	String destination operations (DI offset); extra data storage

Physical Address Generation

Step 1: Shift the segment register LEFT by 4 bits (multiply by 10H).

Step 2: Add the 16-bit Effective Address (offset).

Physical Address = Segment Register × 10H + Effective Address

Worked Examples

Example 1: Code Segment — CS = 348AH, IP = 2359H

```
CS × 10H = 348A0H
Add IP:   + 2359H
-----
PA:      36BF9H
```

Example 2: Data Segment — DS = 2000H, DI = 5A00H

```
DS × 10H = 20000H
Add DI:   + 5A00H
-----
PA:      25A00H
```

Example 3: Stack Segment — SS = 4000H, SP = 64A0H

```
SS × 10H = 40000H
Add SP:   + 64A0H
-----
PA:      464A0H
```

Advantages of Segmentation

- Allows 20-bit addressing using 16-bit registers.

- Each segment can be independently placed and protected.
- Programs can be relocated by changing only segment base registers.
- Segments can overlap, allowing shared data between modules.
- Separate code and data segments improve program modularity.

Q10. What is an addressing mode? List out any 6 addressing modes and explain with examples.

[10 Marks]

Definition

Addressing Mode refers to the method by which the operand of an instruction is specified or located. It tells the CPU **where to find the data**. The Effective Address (EA) is computed from: **EA = Base Register + Index Register + Displacement** (any combination).

1. Immediate Addressing Mode

Description: The operand data is **embedded directly in the instruction** itself. No separate memory access is needed.

Examples:

```
MOV AX, 1234H ; AX=1234H (1234H is immediate data)
```

```
ADD AL, 35H ; AL=AL+35H
```

```
MOV CX, 100 ; CX=100
```

***Note:** Fast (no extra memory read). Data size is fixed in instruction.*

2. Register Addressing Mode

Description: The operand is stored in one of the **internal CPU registers**. No memory access needed.

Examples:

```
MOV AX, CX ; AX=CX (both operands are registers)
```

```
ADD BX, SI ; BX=BX+SI
```

```
INC DX ; DX=DX+1
```

***Note:** Fastest mode (registers are internal to CPU). No memory access.*

3. Direct Addressing Mode

Description: The **16-bit effective address** is given directly in the instruction as a constant. CPU accesses the memory location at DS:EA.

Examples:

```
MOV AX, [2000H] ; AX=DS:[2000H] (PA=DS×10H+2000H)
```

```
MOV [3000H], BX ; DS:[3000H]=BX
```

```
MOV AX, PRICE ; PRICE variable; EA=PRICE offset
```

***Note:** One memory access to get data.*

4. Register Indirect Addressing Mode

Description: The **effective address is held in a register** (BX, SI, DI, or BP). The register acts as a pointer to memory.

Examples:

```
MOV AX, [BX] ; AX=DS:[BX] (EA=contents of BX)
```

```
MOV [SI], AL ; DS:[SI]=AL
```

```
ADD CX, [DI] ; CX=CX+DS:[DI]
```

Note: BX/SI/DI use DS; BP uses SS.

5. Based Addressing with Displacement

Description: EA = Base Register (BX or BP) + 8/16-bit displacement. Useful for accessing structure fields.

Examples:

```
MOV AX, [BX+10H] ; EA=BX+10H; AX=DS:[BX+10H]
```

```
MOV AX, [BP+6] ; EA=BP+6; AX=SS:[BP+6]
```

```
ADD AL, [BX+4] ; EA=BX+4
```

Note: Useful for records/structs. BP is used for stack frame parameters.

6. Based Indexed Addressing with Displacement

Description: EA = Base Register + Index Register + optional Displacement. Most powerful mode, ideal for 2D arrays.

Examples:

```
MOV AX, [BX+SI+100H] ; EA=BX+SI+100H; AX=DS:[EA]
```

```
MOV [BX+DI+50H], CL ; DS:[BX+DI+50H]=CL
```

```
ADD AX, [BP+DI+20H] ; AX=AX+SS:[EA]
```

Note: Most versatile mode. Base: BX or BP. Index: SI or DI. Cannot mix two bases or two indexes.

Q11. With examples, explain shift and rotate instructions available in 8086.

[10 Marks]

This question covers the same instructions as Q8 in detail with additional bit-level examples.

Classification of Shift/Rotate Instructions

Category	Instructions	Key Property
Logical Shift	SHL, SHR	Fill with 0; treat data as unsigned
Arithmetic Shift	SAL, SAR	SAL=SHL; SAR fills with sign bit; treat as signed
Pure Rotate	ROL, ROR	No bit lost; MSB/LSB wraps around
Rotate thru Carry	RCL, RCR	CF is part of rotate; effectively 9-bit (byte) rotate

SHL AL, 1	CF : x	10110001B	→	01100010B	CF : 1	Multiply by 2
SHR AL, 1	CF : x	10110001B	→	01011000B	CF : 1	Unsigned divide by 2
SAR AL, 1	CF : x	10110001B	→	11011000B	CF : 1	Signed divide by 2
ROL AL, 1	CF : x	10110001B	→	01100011B	CF : 1	Bit cycling
ROR AL, 1	CF : x	10110001B	→	11011000B	CF : 1	Bit cycling
RCL AL, 1	CF : 0	10110001B	→	01100010B	CF : 1	9-bit left shift
RCR AL, 1	CF : 1	10110001B	→	11011000B	CF : 1	9-bit right shift

Key Rules

- Count = 1: specify directly in instruction (SHL AL, 1)
- Count > 1: load count into CL register first (MOV CL, 4 / SHL AX, CL)
- SHL and SAL are completely identical — same opcode, same operation
- SHR fills 0 into MSB (logical) — use for UNSIGNED numbers
- SAR copies sign bit into MSB (arithmetic) — use for SIGNED numbers

Q12. Explain with example the working of the instructions: i) AAM ii) DAA

[10 Marks]

Background — BCD and ASCII Arithmetic

The 8086 supports BCD (Binary Coded Decimal) arithmetic. **Packed BCD:** Two decimal digits per byte. E.g., 59 decimal = 59H. **Unpacked BCD:** One decimal digit per byte (upper nibble = 0).

i) AAM — ASCII Adjust AX after Multiply

Purpose: Adjusts the result of multiplying two single-digit unpacked BCD numbers to produce the correct two-digit unpacked BCD result in AX.

- AAM divides AL by 10: quotient (tens digit) goes to AH, remainder (units digit) stays in AL.
- The implied operation: $AH = AL \div 10$; $AL = AL \bmod 10$

Example 1: $5 \times 9 = 45$

```
MOV AL, 05H      ; BCD 5
MOV BH, 09H      ; BCD 9
MUL BH           ; AX = 05H × 09H = 002DH = 45 decimal
AAM              ; AH = 45 ÷ 10 = 4, AL = 45 mod 10 = 5
                  ; AX = 0405H = Unpacked BCD 45 (5 × 9 = 45) ✓
```

Example 2: $7 \times 8 = 56$

```
MOV AL, 07H
MOV BH, 08H
MUL BH           ; AX = 0038H = 56 decimal
AAM              ; AH = 5, AL = 6 → AX = 0506H = Unpacked BCD 56 ✓
```

ii) DAA — Decimal Adjust AL after Addition

Purpose: After adding two packed BCD numbers using ADD, DAA adjusts the result in AL to produce the correct packed BCD result.

Rule 1: If $(AL \text{ AND } 0FH) > 9$ OR $AF = 1 \rightarrow$ Add 06H to AL

Rule 2: If (upper nibble of AL) > 9 OR $CF = 1 \rightarrow$ Add 60H to AL; set CF

Example 1: $59 \text{ BCD} + 35 \text{ BCD} = 94 \text{ BCD}$

```
MOV AL, 59H      ; Packed BCD 59
MOV BL, 35H      ; Packed BCD 35
ADD AL, BL       ; AL = 8EH (binary, not valid BCD!)
DAA              ; Lower nibble E > 9 → Add 06H: AL = 8EH + 06H = 94H
                  ; Upper nibble 9: not > 9, CF = 0 → no 60H needed
                  ; Result: AL = 94H = Packed BCD 94 ✓ (59 + 35 = 94)
```

Example 2: $88 \text{ BCD} + 49 \text{ BCD} = 137 \text{ BCD}$

```
MOV AL, 88H      ; Packed BCD 88
MOV BL, 49H      ; Packed BCD 49
```

```
ADD AL, BL      ; AL = D1H, CF=0, AF=1
DAA
; AF=1 → Add 06H: AL = D7H
; Upper nibble D > 9 → Add 60H: AL = 37H, CF=1
; Result: CF=1 (hundreds 1), AL=37H → BCD 137 ✓
```

Q13. Write an ALP to read a character from the keyboard and display it with proper messages.

[10 Marks]

DOS Interrupt Functions Used

AH value	INT 21H Function	Operation
09H	Display String	Displays a \$ terminated string. DX = offset of string
01H	Read Character	Reads one character from keyboard; echoes it; stores in AL
02H	Display Character	Displays character in DL register on screen
4CH	Terminate Program	Returns control to DOS

Complete ALP

```
DATA SEGMENT
    MSG1 DB "Enter a character: $"
    MSG2 DB 0DH, 0AH, "You entered : $"
    MSG3 DB 0DH, 0AH, "Thank you!$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA DX, MSG1
    MOV AH, 09H
    INT 21H
    MOV AH, 01H
    INT 21H                ; AL = character entered
    MOV BL, AL             ; Save character
    LEA DX, MSG2
    MOV AH, 09H
    INT 21H
    MOV DL, BL
    MOV AH, 02H
    INT 21H
    LEA DX, MSG3
    MOV AH, 09H
    INT 21H
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Sample Output:

Enter a character: **A**

You entered : A

Thank you!

Q14. Explain the different general-purpose and flag registers of the 8086 microprocessor with a neat diagram.

[10 Marks]

A) General-Purpose Data Registers

Register	16-bit	8-bit High	8-bit Low	Primary Special Use
Accumulator	AX	AH	AL	MUL, DIV, IN, OUT; 32-bit result = DX:AX
Base	BX	BH	BL	Memory addressing: [BX], [BX+SI], etc.
Count	CX	CH	CL	Loop counter (LOOP), shift count (CL for count > 1)
Data	DX	DH	DL	32-bit MUL/DIV (DX:AX); 16-bit I/O port address (DX)

B) Pointer & Index Registers

Register	Name	Default Segment	Function
SP	Stack Pointer	SS	Points to top of stack; auto-decremented by PUSH (SP-2), auto-incremented by POP (SP+2)
BP	Base Pointer	SS	Used to access stack frame data. Can also be offset in SS segment
SI	Source Index	DS	Used as source pointer in string operations and indexed addressing
DI	Destination Index	ES/DS	Used as destination pointer in string operations
IP	Instruction Pointer	CS	Holds offset of next instruction; automatically updated by BIU

C) Flag Register

15	14	13	12	11 OF	10 DF	9 IF	8 TF	7 SF	6 ZF	5	4 AF	3	2 PF	1	0 CF
—	—	—	—	OF	DF	IF	TF	SF	ZF	—	AF	—	PF	—	CF

Status Flags — Set automatically by arithmetic/logic results:

- **CF (Carry, bit 0):** Set when addition generates carry out of MSB, or subtraction needs borrow. Also used in multi-byte arithmetic.
- **PF (Parity, bit 2):** Set when the lower 8 bits of the result contain an EVEN number of 1-bits.
- **AF (Auxiliary Carry, bit 4):** Set when there is a carry from bit 3 to bit 4. Used by DAA and AAA for BCD arithmetic correction.
- **ZF (Zero, bit 6):** Set when the result is exactly zero. Used with JZ/JNZ conditional jumps.
- **SF (Sign, bit 7):** Set when the MSB of the result is 1, indicating a negative result in signed arithmetic.
- **OF (Overflow, bit 11):** Set when a signed arithmetic operation produces a result too large or too small for the destination.

Control Flags — Set deliberately by programmer:

- **DF (Direction, bit 10):** CLD → DF=0 → SI/DI auto-INCREMENT in string ops. STD → DF=1 → SI/DI auto-DECREMENT.

- **IF (Interrupt Enable, bit 9):** STI → IF=1 → accepts external maskable interrupts. CLI → IF=0 → interrupts ignored.
- **TF (Trap, bit 8):** When set, processor single-steps: executes one instruction then generates a Type 1 interrupt. Used by debuggers.

Q15. Explain the different memory addressing modes of the 8086 microprocessor with an example.

[10 Marks]

Introduction

Memory addressing modes are those where the operand resides **in memory**. The CPU must compute a 20-bit physical address. The **Effective Address (EA)** is computed from: **Base Register + Index Register + Displacement**.

1. Direct Addressing

Description: The 16-bit offset (EA) is given directly in the instruction.

```
MOV AX, [2000H] ; EA = 2000H, PA = DS×10H + 2000H
MOV [PRICE], AL ; PRICE offset is EA
```

2. Register Indirect

Description: EA is held in a single register: BX, SI, DI (DS) or BP (SS).

```
MOV AX, [BX] ; EA = BX
MOV [SI], DX ; EA = SI
MOV CX, [BP] ; EA = BP, uses SS
```

3. Based with Displacement

Description: EA = Base Register (BX or BP) + 8/16-bit Displacement.

```
MOV AX, [BX + 04H] ; EA = BX + 4
MOV AX, [BP + 6] ; EA = BP + 6 (stack frame)
```

4. Indexed with Displacement

Description: EA = Index Register (SI or DI) + 8/16-bit Displacement.

```
MOV AX, [SI + 20H] ; EA = SI + 20H
MOV BX, [DI + 8] ; EA = DI + 8
```

5. Based Indexed

Description: EA = Base Register + Index Register.

```
MOV AX, [BX + SI] ; EA = BX + SI
MOV [BX + DI], CX ; EA = BX + DI
```

6. Based Indexed + Displacement

Description: EA = Base + Index + Displacement. Most powerful mode.

```
MOV AX, [BX + SI + 100H] ; EA = BX + SI + 100H
ADD CX, [BX + DI + 3456H] ; EA = BX + DI + 3456H
```

Rules for Memory Addressing

- Base registers: **BX or BP only**
- Index registers: **SI or DI only**

- **[BP]** uses **SS** as default segment; all others use **DS**
- Cannot combine two base registers: **[BX+BP]** is **INVALID**
- Cannot combine two index registers: **[SI+DI]** is **INVALID**
- Cannot have two memory operands: **ADD [5000H],[3000H]** is **INVALID**

Q16. Explain the following 8086 instructions with an example for each: i) DAA ii) AAA

[10 Marks]

i) DAA — Decimal Adjust AL after BCD Addition

Type: Decimal/BCD Arithmetic | **Operand:** Implied — always works on AL

Algorithm:

```
IF ((AL AND 0FH) > 9) OR (AF = 1) THEN AL = AL + 06H
IF (upper nibble > 9) OR (CF = 1) THEN AL = AL + 60H; CF = 1
```

Example: Packed BCD 59 + 35

```
MOV AL, 59H
MOV BL, 35H
ADD AL, BL    ; AL = 8EH (wrong BCD)
DAA
; Lower nibble E > 9 → +06H → AL=94H
; Upper 9, not >9 → no 60H
; Result: 94H = BCD 94 ✓
```

ii) AAA — ASCII Adjust AL after Addition

Type: ASCII/Unpacked BCD | **Operand:** Implied — works on AL and AH

Algorithm:

```
IF ((AL AND 0FH) > 9) OR (AF = 1) THEN
    AL = AL + 06H; AH = AH + 01H; AF = 1; CF = 1
ELSE AF = 0; CF = 0
AL = AL AND 0FH
```

Example 1: ASCII '5' + ASCII '9' = 14

```
MOV AH, 00H
MOV AL, 35H ; ASCII '5'
MOV BL, 39H ; ASCII '9'
ADD AL, BL  ; AL = 6EH
AAA
; Lower nibble E > 9 → +06H → 74H; AH=01H
; AL=74H AND 0FH=04H
; AH=01, AL=04 → Unpacked BCD 14 ✓
```

Example 2: ASCII '3' + ASCII '4' = 7 (no carry)

```
MOV AH, 00H
MOV AL, 33H ; ASCII '3'
MOV BL, 34H ; ASCII '4'
ADD AL, BL  ; AL = 67H
AAA
; Lower nibble 7, not > 9 → no correction
; AL = 67H AND 0FH = 07H → Unpacked BCD 7 ✓
```

Q17. AX=1234H, BX=5678H, SI=ABCDH, DI=CDEFH, CS=3456H, IP=789AH, DS=ES=4567H. Identify addressing modes and work out addresses for: i)MOV[BX],AX ii)MOV[BX+DI+3456H],9ABCH iii)MOV AX,[9ABC] iv)MOV CX,[SI+BX]

[10 Marks]

Given Register Values

Register	Value	Register	Value	Register	Value
AX	1234H	BX	5678H	SI	ABCDH
DI	CDEFH	CS	3456H	IP	789AH
DS	4567H	ES	4567H	—	—

PA = Segment Register × 10H + EA | All use DS = 4567H (except BP which uses SS)

i) MOV [BX], AX

Mode: Register Indirect Addressing

EA: EA = BX = 5678H

PA: $PA = DS \times 10H + EA = 45670H + 5678H = 4BCE8H$

ii) MOV [BX+DI+3456H], 9ABCH

Mode: Based Indexed Addressing with Displacement

EA: $EA = BX + DI + 3456H = 5678H + CDEFH + 3456H = 57BDH$ (lower 16 bits)

PA: $PA = DS \times 10H + EA = 45670H + 57BDH = 4AE2DH$

iii) MOV AX, [9ABC]

Mode: Direct Addressing

EA: EA = 9ABCH (16-bit constant directly in instruction)

PA: $PA = DS \times 10H + EA = 45670H + 9ABCH = 4F12CH$

iv) MOV CX, [SI+BX]

Mode: Based Indexed Addressing

EA: $EA = SI + BX = ABCDH + 5678H = 0245H$ (lower 16 bits)

PA: $PA = DS \times 10H + EA = 45670H + 0245H = 458B5H$

Summary

Instruction	Mode	EA	Physical Address
MOV [BX], AX	Register Indirect	5678H	4BCE8H
MOV [BX+DI+3456H], 9ABCH	Based Indexed + Displacement	57BDH	4AE2DH

MOV AX, [9ABC]	Direct	9ABCH	4F12CH
MOV CX, [SI+BX]	Based Indexed	0245H	458B5H

Q18. Explain the advantages of segmentation in 8086. Illustrate with an example how the physical address is generated for the code segment.

[10 Marks]

Advantages of Segmentation

- 1. Extended Addressing:** Uses 16-bit segment + 16-bit offset to generate 20-bit physical addresses, allowing 1 MB access.
- 2. Memory Protection:** Code, data, and stack in separate segments prevents accidental overwriting.
- 3. Code Relocation:** A program can be moved by updating only segment register values; code itself (offsets) unchanged.
- 4. Modular Programming:** Different modules can each occupy separate segments, enabling independent development.
- 5. Efficient Stack Management:** SS:SP provides automatic stack management; stack is isolated from code and data.
- 6. Sharing of Code/Data:** Multiple programs can share the same code segment, saving memory in multi-tasking.
- 7. Simplified Linking:** The linker combines separately assembled modules by adjusting segment base addresses.

Physical Address — Code Segment Example

Code Segment PA = CS × 10H + IP

Example 1: CS = 348AH, IP = 2359H

```
CS × 10H = 348A0H
Add IP:   + 2359H
-----
PA:      36BF9H
```

Example 2: CS = 1000H, IP = A000H

```
CS × 10H = 10000H
Add IP:   + A000H
-----
PA:      1A000H
```


Q19. Explain the working of the Bus Interface Unit (BIU) of the 8086 architecture with a diagram.

[10 Marks]

Introduction to BIU

The **Bus Interface Unit (BIU)** handles all **external bus communications**. It acts as the interface between the internal processor logic and the external system buses.

Components of the BIU

Component	Size	Function
Segment Registers (CS,DS,SS,ES)	4 × 16-bit	Hold base addresses of the four active memory segments
Instruction Pointer (IP)	16-bit	Holds the offset of the next instruction within CS
6-Byte Instruction Queue	6 bytes FIFO	Pre-fetches and stores up to 6 instruction bytes from memory
Address Adder	20-bit adder	Computes physical address: Segment × 10H + Offset
Bus Control Logic	Control signals	Generates MEMR, MEMW, IOR, IOW, ALE, etc.

Working of the BIU — Step by Step

Step 1: Instruction Fetch: BIU reads CS, adds IP to form 20-bit physical address, places on address bus.

Step 2: Bus Cycle Initiation: BIU puts address on 20-bit address bus and asserts ALE (Address Latch Enable).

Step 3: Data Read: BIU activates MEMR control signal; memory places instruction byte on data bus.

Step 4: Queue Loading: BIU stores fetched bytes in the 6-byte FIFO queue; IP is incremented.

Step 5: Queue Management: BIU continuously tries to keep the queue full — initiates new fetch if 2+ empty bytes.

Step 6: EU Reads from Queue: EU reads the next instruction from the front of the queue (pipelining).

Step 7: Data Operand Access: When EU needs a memory operand, it sends EA to BIU; BIU computes PA and performs bus cycle.

Step 8: Branch Handling: On jump/CALL, BIU flushes the queue (prefetched data invalid) and refills from new address.

Pipelining Benefit: BIU pre-fetches instructions independently from EU execution. Queue hides memory latency, increasing throughput by ~30%.

**Q20. Explain the following assembler directives: i)DD ii)EVEN
iii)INCLUDE iv)OFFSET v)DB**

[10 Marks]

i) DD — Define Double Word

Description: DD allocates and optionally initializes **4 bytes (32 bits)** of memory storage. Used to declare 32-bit variables.

```
BIG_NUM DD 12345678H ; 4 bytes initialized
FAR_PTR DD ?          ; 4 bytes uninitialized
TABLE   DD 10 DUP(0)  ; Array of 10 double-words
```

KEY POINT: 1 double word = 4 bytes = 32 bits. Stored in little-endian order.

ii) EVEN — Align to Even Address

Description: EVEN instructs the assembler to align the next item to an **even memory address**. Word (16-bit) access at an even address requires only ONE bus cycle; odd address requires TWO (slower).

```
DATA SEGMENT
A1 DB 9 DUP(?)      ; 9 bytes (address 0000-0008H)
EVEN                ; If LC = 0009H → insert NOP → LC = 000AH
B1 DW 100 DUP(0)    ; Word array now starts at even 000AH
DATA ENDS
```

KEY POINT: EVEN inserts 1-byte NOP (90H) if needed to reach an even address.

iii) INCLUDE — Include Source File

Description: INCLUDE inserts the entire contents of an external file at the point of the directive.

```
INCLUDE MACROS.ASM ; Insert all of MACROS.ASM here
INCLUDE C:\MYLIB\IO.ASM ; Full path allowed
INCLUDE STDDEF.INC ; Standard definitions
```

KEY POINT: Like #include in C. Changes in the included file reflect everywhere it is used.

iv) OFFSET — Get Offset of Variable

Description: OFFSET returns the **16-bit offset address** of a variable, label, or procedure from the start of its segment.

```
MOV SI, OFFSET PRICES ; SI = offset of PRICES array
MOV DX, OFFSET MSG    ; DX = address of MSG string
LEA SI, PRICES         ; Equivalent to MOV SI, OFFSET PRICES
```

KEY POINT: OFFSET gives the ADDRESS of the variable, not its value. Compare: MOV AX, VAR gets value; MOV AX, OFFSET VAR gets address.

v) DB — Define Byte

Description: DB allocates **one byte (8 bits)** per item and optionally initializes it. Most commonly used data definition directive.

```
PRICE DB 49H, 98H, 29H ; Array of 3 bytes
NAME  DB "HELLO"        ; 5 ASCII bytes
BUFFER DB 100 DUP(?)    ; 100 uninitialized bytes
```

```
NEWLINE DB 0DH, 0AH, "$" ; CR, LF, $
```

KEY POINT: *DUP operator allows repeating a value multiple times.*

Q21. Develop an ALP to search a key element in a list of n 8-bit numbers using Linear Search. Display position if found; else display "Element Not Found".

[10 Marks]

Complete ALP with Display

```
DATA SEGMENT
    ARR      DB  25H, 10H, 45H, 30H, 60H, 15H
    N        DB  06H
    KEY       DB  45H
    MSG1      DB  0DH, 0AH, "Element Found at Position: $"
    MSG2      DB  0DH, 0AH, "Element Not Found!$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA SI, ARR
    MOV CL, N
    MOV CH, 00H
    MOV AL, KEY
    MOV BL, 01H
SEARCH_LOOP:
    CMP AL, [SI]
    JE  ELEMENT_FOUND
    INC SI
    INC BL
    DEC CX
    JNZ SEARCH_LOOP
ELEMENT_NOT_FOUND:
    LEA DX, MSG2
    MOV AH, 09H
    INT 21H
    JMP EXIT
ELEMENT_FOUND:
    LEA DX, MSG1
    MOV AH, 09H
    INT 21H
    MOV DL, BL
    ADD DL, 30H
    MOV AH, 02H
    INT 21H
EXIT:
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Output: Element Found at Position: 3 (45H is at index 3 of the array)

Q22. How do various Shift Instructions work? Differentiate between arithmetic and logical shift instructions. Explain with examples.

[10 Marks]

Detailed Comparison — Logical vs Arithmetic Shifts

Feature	Logical Shift (SHL/SHR)	Arithmetic Shift (SAL/SAR)
Number Type	For UNSIGNED numbers	For SIGNED numbers (2s complement)
Left Shift	SHL: 0 fills LSB, MSB→CF	SAL: identical to SHL in every way
Right Shift	SHR: 0 fills MSB, LSB→CF	SAR: SIGN BIT copies to MSB, LSB→CF
Effect ×2	SHL multiplies unsigned by 2	SAL multiplies signed by 2
Effect ÷2	SHR divides unsigned by 2	SAR divides signed by 2
Sign Handling	Ignores sign — always inserts 0	SAR PRESERVES sign — propagates MSB

Example 1: SHL — Multiply by 4

```
MOV AL, 00001100B ; AL = 0CH = 12
MOV CL, 02H
SHL AL, CL ; 12 × 4 = 48 = 30H
; Before: 00001100 → After shift 2: 00110000 ✓
```

Example 2: SHR — Divide Unsigned by 2

```
MOV BL, 10011000B ; BL = 98H = 152 (unsigned)
SHR BL, 1 ; 152 ÷ 2 = 76
; Before: 10011000 → After: 01001100 = 76 ✓
```

Example 3: SAR — Divide SIGNED by 2 (NEGATIVE)

```
MOV AL, 11001000B ; AL = C8H = -56 (signed)
SAR AL, 1 ; -56 ÷ 2 = -28
; Before: 11001000 → After: 11100100 = E4H = -28 (MSB PRESERVED!) ✓
```

Compare: SHR on same negative number (WRONG for signed!)

```
MOV AL, 11001000B ; AL = C8H = -56 signed
SHR AL, 1 ; Before: 11001000 → After: 01001100
; Result: +76 – WRONG for signed division!
```

Rule: Use SHR for unsigned, SAR for signed numbers. SHL = SAL for left shifts.

**Q23. Develop an ALP with procedures to: i) Read a string from keyboard
ii) Display the string iii) Display the length of the string**

[10 Marks]

Complete ALP with Three Procedures

```
DATA SEGMENT
    PROMPT DB "Enter a string: $"
    NEWLN DB 0DH, 0AH, "$"
    MSG_DSP DB "String entered: $"
    MSG_LEN DB "Length: $"
    BUFFER DB 50
           DB 0
           DB 50 DUP(?)
DATA ENDS
```

```
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
```

```
READ_STRING PROC NEAR
    LEA DX, PROMPT
    MOV AH, 09H
    INT 21H
    LEA DX, BUFFER
    MOV AH, 0AH
    INT 21H
    RET
READ_STRING ENDP
```

```
DISP_STRING PROC NEAR
    LEA DX, NEWLN
    MOV AH, 09H
    INT 21H
    LEA DX, MSG_DSP
    MOV AH, 09H
    INT 21H
    MOV CL, BUFFER+1
    MOV CH, 00H
    LEA SI, BUFFER+2
```

```
DISP_LOOP:
    MOV DL, [SI]
    MOV AH, 02H
    INT 21H
    INC SI
    LOOP DISP_LOOP
    RET
```

```
DISP_STRING ENDP
```

```
DISP_LENGTH PROC NEAR
    LEA DX, NEWLN
    MOV AH, 09H
    INT 21H
    LEA DX, MSG_LEN
    MOV AH, 09H
```

```
    INT 21H
    MOV DL, BUFFER+1
    ADD DL, 30H
    MOV AH, 02H
    INT 21H
    RET
DISP_LENGTH ENDP

START:
    MOV AX, DATA
    MOV DS, AX
    CALL READ_STRING
    CALL DISP_STRING
    CALL DISP_LENGTH
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Sample Run:

Enter a string: HELLO

String entered: HELLO

Length: 5

Q24. Explain the flag register along with the register format with respect to Intel 8086.

[10 Marks]

Comprehensive coverage of all 9 flags with format and examples.

Flag Register Format (16-bit)

15	14	13	12	11 OF	10 DF	9 IF	8 TF	7 SF	6 ZF	5	4 AF	3	2 PF	1	0 CF
—	—	—	—	OF	DF	IF	TF	SF	ZF	—	AF	—	PF	—	CF

Detailed Description of All 9 Flags

CF — Carry Flag (bit 0) (Status Flag)

Description: Set when addition generates carry out of MSB, or subtraction needs borrow. Also used in multi-precision arithmetic.

```
0FFH + 01H = 100H → CF=1
ADC AX, BX (adds carry to extend precision)
```

PF — Parity Flag (bit 2) (Status Flag)

Description: Set when lower 8 bits of result have EVEN number of 1-bits. Used in data communication error checking.

```
00110011B (four 1s → even) → PF=1
01110001B (four 1s → even) → PF=1
```

AF — Auxiliary Carry Flag (bit 4) (Status Flag)

Description: Set when carry from bit 3 to bit 4. Used by DAA and AAA for BCD arithmetic correction.

```
0FH + 01H = 10H → carry bit3 to bit4 → AF=1
```

ZF — Zero Flag (bit 6) (Status Flag)

Description: Set when result is exactly zero. Used with JZ/JNZ conditional jumps.

```
MOV AX, 5 / SUB AX, 5 → AX=0 → ZF=1
```

SF — Sign Flag (bit 7) (Status Flag)

Description: Set when MSB of result is 1, indicating negative result in signed arithmetic.

```
-1 = FFH = 11111111 → MSB=1 → SF=1
```

OF — Overflow Flag (bit 11) (Status Flag)

Description: Set when signed arithmetic overflows: result too large (positive) or too small (negative) for destination.

```
7FH + 01H = 80H (127+1=-128 signed) → OF=1
```

DF — Direction Flag (bit 10) (Control Flag)

Description: Controls string operation direction. CLD → DF=0 → SI/DI auto-increment. STD → DF=1 → auto-decrement.

```
CLD ; SI/DI increment in MOVSB
STD ; SI/DI decrement in MOVSB
```

IF — Interrupt Enable Flag (bit 9) (Control Flag)

Description: STI sets IF (INTR interrupts accepted). CLI clears IF (INTR interrupts ignored). NMI always accepted.

```
CLI ; Disable interrupts (critical section)
STI ; Re-enable interrupts
```

TF — Trap Flag (bit 8) (Control Flag)

Description: When set, processor single-steps: executes one instruction, generates Type 1 interrupt. Used by debuggers.

```
Set via PUSHF / OR / POPF to enable single-step debugging
```

Q25. Develop an ALP to reverse a given string and check whether it is a palindrome or not.

[10 Marks]

Algorithm

A string is a palindrome if it reads the same forwards and backwards (e.g., "MADAM"). Compare front and back characters working inward; any mismatch → NOT palindrome.

Complete ALP

```
DATA SEGMENT
    STR1    DB  "MADAM"
    LEN     DB  05H
    MSG_YES DB  0DH,0AH,"PALINDROME!$"
    MSG_NO  DB  0DH,0AH,"NOT A PALINDROME!$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA SI, STR1          ; SI → first character
    MOV BL, LEN
    MOV BH, 00H
    MOV DI, SI
    ADD DI, BX
    DEC DI                ; DI → last character
    MOV CL, LEN
    MOV CH, 00H
    SHR CX, 1             ; CX = LEN / 2 comparisons
COMPARE:
    MOV AL, [SI]
    MOV AH, [DI]
    CMP AL, AH
    JNE NOT_PALINDROME
    INC SI
    DEC DI
    LOOP COMPARE
IS_PALINDROME:
    LEA DX, MSG_YES
    MOV AH, 09H
    INT 21H
    JMP DONE
NOT_PALINDROME:
    LEA DX, MSG_NO
    MOV AH, 09H
    INT 21H
DONE:
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Trace — "MADAM"			
Iteration	[SI]	[DI]	CMP Result
1	M (STR[0])	M (STR[4])	Equal → continue
2	A (STR[1])	A (STR[3])	Equal → continue
CX=0	D (STR[2])	D (STR[2])	Loop ends → PALINDROME!
Output for "MADAM": PALINDROME! Output for "HELLO": NOT A PALINDROME!			

Q26. List and explain the various addressing modes available in Intel 8086.

[10 Marks]

Complete List of 8086 Addressing Modes

#	Mode	EA Formula	Example
1	Immediate	Data in instruction	MOV AX, 1234H
2	Register	Operand in register	MOV AX, BX
3	Direct	EA = 16-bit constant	MOV AX, [2000H]
4	Register Indirect	EA = [BX/SI/DI/BP]	MOV AX, [BX]
5	Based w/ Disp	EA = BX/BP + disp	MOV AX, [BX+10H]
6	Indexed w/ Disp	EA = SI/DI + disp	MOV AX, [SI+20H]
7	Based Indexed	EA = Base + Index	MOV AX, [BX+SI]
8	Based Indexed+Disp	EA = Base+Index+Disp	MOV AX, [BX+DI+100H]

Mode 1: Immediate

MOV AL, 25H ; AL=25H

Mode 2: Register

MOV CX, AX ; CX=AX (no memory access)

Mode 3: Direct

MOV AX, [3000H] ; AX=DS:[3000H]

Mode 4: Register Indirect

ADD AX, [SI] ; AX=AX+DS:[SI]

Mode 5: Based w/ Displacement

MOV DX, [BP+4] ; DX=SS:[BP+4] (stack frame)

Mode 6: Indexed w/ Displacement

MOV BL, [DI+2] ; BL=DS:[DI+2]

Mode 7: Based Indexed

MOV AX, [BX+DI] ; AX=DS:[BX+DI]

Mode 8: Based Indexed + Displacement

MOV AX, [BP+SI+100H] ; AX=SS:[BP+SI+100H]

Q27. Explain the following 8086 Assembler Directives with an example for each: i)ASSUME ii)SEGMENT iii)INCLUDE iv)DB

[10 Marks]

i) ASSUME

Syntax: ASSUME CS:CODE_SEG, DS:DATA_SEG, SS:STACK_SEG

Description: Maps logical segment names to segment registers. The assembler uses this to correctly resolve segment references.

```
DATA_SEG SEGMENT
    NUM DB 10H
DATA_SEG ENDS
CODE_SEG SEGMENT
ASSUME CS:CODE_SEG, DS:DATA_SEG
START:
    MOV AX, DATA_SEG
    MOV DS, AX
    MOV AL, NUM    ; Assembler knows NUM is in DS
CODE_SEG ENDS
END START
```

ii) SEGMENT / ENDS

Syntax: segment_name SEGMENT ... segment_name ENDS

Description: SEGMENT marks the beginning of a logical segment. ENDS marks the end. Every SEGMENT must have a matching ENDS.

```
DATA_SEG SEGMENT
    MSG DB "HELLO$"
    NUM1 DW 1234H
    ARR DB 10 DUP(0)
DATA_SEG ENDS

CODE_SEG SEGMENT
ASSUME CS:CODE_SEG, DS:DATA_SEG
START:
    MOV AX, DATA_SEG
    MOV DS, AX
    MOV AH, 4CH
    INT 21H
CODE_SEG ENDS
END START
```

iii) INCLUDE

Syntax: INCLUDE filename.ASM

Description: Inserts the entire contents of an external file at the point of the directive. Like #include in C.

```
; MACROS.INC contains NEWLINE macro definition
CODE_SEG SEGMENT
    INCLUDE MACROS.INC    ; Entire MACROS.INC inserted here
ASSUME CS:CODE_SEG
```

```
START:
    NEWLINE                ; Macro from included file
    MOV AH, 4CH
    INT 21H
CODE_SEG ENDS
END START
```

iv) DB — Define Byte

Syntax: name DB value(s)

Description: Allocates one byte per item. Can initialize with hex, decimal, ASCII, or strings. DUP repeats a value.

```
DATA_SEG SEGMENT
    AGE    DB 25H           ; Single byte
    SCORES DB 85, 90, 78    ; Byte array
    GREET  DB "HELLO"       ; ASCII string (5 bytes)
    MSG    DB "Hi!$"        ; $ terminated string
    BUF    DB 100 DUP(?)    ; 100 uninitialized bytes
    NEWLN  DB 0DH, 0AH, "$" ; CR LF sequence
DATA_SEG ENDS
```

Q28. Explain the role of the following 4 directives: ASSUME, END, PROC, ENDS

[10 Marks]

i) ASSUME

Syntax: ASSUME seg_reg : segment_name [, ...]

Role: Maps logical segment names to physical segment registers so the assembler can resolve segment references during assembly. Does NOT load registers at runtime.

```
ASSUME CS:CODE, DS:DATA, SS:STACK
; Tells assembler:
;   CS register → CODE segment
;   DS register → DATA segment
;   SS register → STACK segment
```

ii) END

Syntax: END [label]

Role: Marks the physical end of the entire assembly source module. The assembler ignores anything after END. Optional label specifies the program entry point.

```
CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START    ; END of source; START = entry point
; Anything below is IGNORED by assembler
```

iii) PROC

Syntax: name PROC [NEAR/FAR] ... name ENDP

Role: PROC defines the start of a procedure (subroutine). ENDP marks the end. NEAR = same segment (16-bit return). FAR = different segment (32-bit return).

```
SORT PROC NEAR
    PUSH AX
    PUSH BX
    ; --- sorting code ---
    POP  BX
    POP  AX
    RET                ; Return to caller
SORT ENDP

; In main code:
CALL SORT            ; Push IP and jump to SORT
```

iv) ENDS

Syntax: segment_name ENDS

Role: Marks the end of a logical segment started with SEGMENT directive. Every SEGMENT must have a matching ENDS with the same name.

```
DATA SEGMENT
    NUM DB 25H
    ARR DB 10 DUP(?)
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```


Q29. Write an ALP to read a character from keyboard and display it. If the entered character is "I", then print a message saying "INTEL 8086 PROCESSOR".

[10 Marks]

Complete ALP

```
DATA SEGMENT
    MSG1 DB "Enter a character: $"
    MSG2 DB 0DH, 0AH, "You entered: $"
    MSG3 DB 0DH, 0AH, "INTEL 8086 PROCESSOR$"
    MSG4 DB 0DH, 0AH, "Program terminated.$"
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX
    LEA DX, MSG1
    MOV AH, 09H
    INT 21H
    MOV AH, 01H
    INT 21H           ; AL = entered character
    MOV BL, AL        ; Save in BL
    LEA DX, MSG2
    MOV AH, 09H
    INT 21H
    MOV DL, BL
    MOV AH, 02H
    INT 21H           ; Display character
    CMP BL, "I"       ; Is it ASCII "I" = 49H?
    JNE NOT_I
    LEA DX, MSG3
    MOV AH, 09H
    INT 21H           ; Display "INTEL 8086 PROCESSOR"
NOT_I:
    LEA DX, MSG4
    MOV AH, 09H
    INT 21H
    MOV AH, 4CH
    INT 21H
CODE ENDS
END START
```

Input	Screen Output
I	Enter a character: I / You entered: I / INTEL 8086 PROCESSOR / Program terminated.
A	Enter a character: A / You entered: A / Program terminated.

Q30. Check whether the following 8086 instructions are valid or not. Explain the reason: i)ADD AX,100H ii)MOV BL,AX iii)ADD CX,1234H iv)MOV DS,5000H v)OUT 9000H,AX vi)ADD [5000H],[3000H]

[10 Marks]

Rules for Valid 8086 Instructions

- Both operands must be the same SIZE (byte or word).
- Segment registers CANNOT be loaded with an immediate value directly.
- OUT fixed port: address must be 0–FFH; ports > FFH require DX register.
- Memory-to-memory operations are NOT allowed.
- IN/OUT: only AL (byte) or AX (word) can be used as accumulator.

i) ADD AX, 100H

■ VALID

Reason: AX is 16-bit, 100H is a 16-bit immediate value. Same size. ADD supports register + immediate.

Fix: No fix needed.

ii) MOV BL, AX

■ INVALID

Reason: BL is 8-bit, AX is 16-bit. SIZE MISMATCH — both operands must be same size.

Fix: Fix: MOV BL, AL (both 8-bit) or MOV BX, AX (both 16-bit)

iii) ADD CX, 1234H

■ VALID

Reason: CX is 16-bit, 1234H is a 16-bit immediate. Same size. Perfectly valid.

Fix: No fix needed.

iv) MOV DS, 5000H

■ INVALID

Reason: Segment registers CANNOT be loaded from an immediate value. Hardware restriction.

Fix: Fix: MOV AX, 5000H then MOV DS, AX

v) OUT 9000H, AX

■ INVALID

Reason: 9000H > FFH. Fixed port address max is FFH. Ports above FFH must use DX register.

Fix: Fix: MOV DX, 9000H then OUT DX, AX

vi) ADD [5000H], [3000H]

■ INVALID

Reason: 8086 DOES NOT support memory-to-memory. Both operands are memory references.

Fix: Fix: MOV AX, [3000H] then ADD [5000H], AX

Quick Summary

#	Instruction	Valid?	Reason
i	ADD AX, 100H	■ Valid	Both 16-bit operands

ii	MOV BL, AX	■ Invalid	Size mismatch: BL=8-bit, AX=16-bit
iii	ADD CX, 1234H	■ Valid	Both 16-bit; register + immediate allowed
iv	MOV DS, 5000H	■ Invalid	Cannot load segment register with immediate
v	OUT 9000H, AX	■ Invalid	9000H > FFH; must use DX for ports > FFH
vi	ADD [5000H],[3000H]	■ Invalid	Two memory operands not allowed

**END OF QUESTION BANK — ALL 30 QUESTIONS ANSWERED IN
DETAIL**

Good Luck with Your Examination!